7 寸 RGB-LCD 屏 + 电容触摸 基本性能测试程序

> 目标硬件：正点原子 **达芬奇-lite**（`xc7a35tfgg484-2`）+ 正点原子 **ATK-7016 系列 7 寸 1024×600 IPS 电容屏**（40P 0.5mm FPC，触摸 IC GT911 / FT5206 / FT5446）。 本程序**完全独立于仓库里既有的编码器工程**：自带顶层、自带约束、自带测试台， 只借用同一块板子的引脚号，不读也不改 `rtl/`、`constr/enc_top.xdc`。

---

1. 这个程序要回答什么问题

"屏幕上电亮不亮"是一个只能靠眼睛的判断，但绝大多数接线/时序故障其实可以提前 自动判定。本程序把测试拆成**两层**：

层	靠什么判定	能发现什么
可自动判定（仿真 + 上板遥测）	屏 ID 读数、行/场周期与有效像素数、触摸 IC 是否锁定、IIC 传输/错误计数	线序错、像素时钟错、时序参数错、触摸不通信、IIC 地址错
只能靠眼睛（10 幅画面）	彩条、纯色、灰阶、1 像素棋盘、网格、运动块、触控十字	颜色通道接反、偏色、坏点、亮度不均、坐标映射错、刷新撕裂

第二层里有两幅**专门为"性能"设计的画面**，不是装饰：

- **画面 3（1 像素棋盘）** —— 最狠的像素级测试。颗粒糊成一片说明像素时钟相位/ 采样沿不对（串行 RGB 时代才会遇到，但 PCB 走线偏斜、pclk 与数据错沿也会这样）。
- **画面 9（10×10 触控响应格）** —— 手指走遍全屏，看是否每个格子都能被点亮、 多点并发时是否两个不同颜色的格子同时亮。常见故障是 x/y 少一位（只有半边屏 有响应）或某轴反向。

---

2. 文件清单

```
lcd_test/
├── rtl/
│   ├── lcd_rgb_test_top.v      顶层：拼装 ID 读取 + 时序 + 画面 + 触摸 + 遥测
│   ├── lcd_timing_gen.v        DE 模式时序发生器（运行时可变 H/V 参数 + 像素使能）
│   ├── lcd_test_pattern.v      10 幅测试画面 + 5×7 点阵字符渲染
│   ├── touch_i2c_ctrl.v        触摸探测/轮询序列器（GT911 与 FT5x06 双协议）
│   ├── i2c_master_seq.v        字节级 IIC 主机（开漏，支持重复 START 与 STOP+START）
│   └── lcd_test_support.v      小模块集合：复位同步 / 按键消抖 / 屏 ID 读取 /
│                               IIC 引脚三态包装 / UART 发送 / 16 字节帧发送
├── constr/
│   └── lcd_test.xdc            34 个引脚的约束 + 输出延时/虚假路径
├── sim/
│   ├── tb_lcd_timing.v         时序发生器自检（含帧周期回归）
│   ├── tb_uart_frame.v         ★ UART 一帧 16 字节的 8N1 帧级自检（逐位采样解码）
│   ├── tb_touch_i2c.v          IIC 触摸自检（GT911 / FT5206 / 无从机 三种配置）
│   ├── iic_slave_model.v       行为级 IIC 从机模型（GT911 与 FT5206 寄存器表）
│   ├── tb_lcd_rgb_top.v        集成自检（ID→时序→触摸→按键→UART）
│   └── iobuf_stub.v            仿真用 IOBUF 桩（综合时用真实原语）
└── tools/
    ├── sim_lcd_test.tcl        跑五个测试台（UART 帧级 + GT911 与 FT5206 各一遍）
    ├── synth_lcd_test.tcl      综合验证 + 双向 IOBUF 检查
    ├── impl_lcd_test.tcl       实现 + 生成位流（上板用）
    ├── program_lcd_test.tcl    ★ 经 JTAG 下载位流（只写易失配置，不动 SPI flash）
    └── lcd_uart_check.py       ★ 读 UART 遥测并解码（115200 8N1，需 pyserial）
```

---

3. 引脚与硬件前提

引脚全部来自《达芬奇-lite 之 FPGA 开发指南》表 2.1.1，并与板级原理图 `Davinci_Lite_1V0.pdf` 第 13 页 `J7(RGB_TFTLCD)` 的矢量几何逐条核对过 （30 个 `LCD_*` 网络，**0 个不一致**），详见 `docs/RGB_LCD_wiring_check.md`。

3.1 LCD 侧（J7，40P 0.5mm FPC)

信号	球号	说明
`lcd_rgb[23:16]`	AA6 AB6 Y8 AB7 AA8 V8 AB8 V9	红 R7..R0
`lcd_rgb[15:8]`	U6 AB3 Y6 AB5 V7 Y7 W7	绿 G7..G0
`lcd_rgb[7:0]`	W1 Y4 Y1 Y3 AA1 AA4 AB1 V5	蓝 B7..B0
`lcd_pclk`	U1	像素时钟 = 50 MHz（直接取 `sys_clk`）
`lcd_hs` / `lcd_vs` / `lcd_de`	AB2 / T1 / W2	DE 模式：HS/VS 低有效，DE 高=数据有效
`lcd_bl`	Y2	**高电平点亮背光**
`lcd_rst`	R2	低电平复位屏

3.2 触摸侧（同一根 FPC)

信号	球号	说明
`touch_scl`	T4	开漏 IIC（板级网名 `T_SCK`，是沿用电阻屏的老命名）
`touch_sda`	U5	开漏 IIC（板级网名 `T_PEN`）
`touch_rst_n`	W4	触摸 IC 复位，低有效
`touch_int`	T5	触摸中断，低有效（本程序只上报，不依赖它触发）

> ★ `lcd_rgb[23:0]` 是**双向**引脚：屏模块在 `R7/G7/B7` 上挂了 ID 分压电阻， FPGA 必须先让这 24 根线**全部高阻**才能读回型号。本程序的 `data_oe` 负责这件事， 综合后应看到 24 个 `IOBUF` （`synth_lcd_test.tcl` 会打印这个数字）。

3.3 Bank 电压

用到的引脚全部在 **Bank 34**（`sys_clk`/`key_n`/LCD/触摸）与 **Bank 14**（`uart_tx`）， 两处 VCCIO 都是**固定 +3.3V**，不需要拨 `J9/J10` 跳线。

---

4. 屏型号自动识别

上电后先让数据线高阻约 42 ms，采一次 `{B7, G7, R7}`：

`{B7,G7,R7}`	型号	分辨率	本程序选的时序（PCLK)
`000`	ATK-4342	480×272	41/43/480/8，10/12/272/8（12.5 MHz，CE=4)
`010`	**ATK-7016**	**1024×600**	**20/140/1024/160，3/20/600/12（50 MHz，CE=1)**
`001`	ATK-7084	800×480	128/88/800/40，2/33/480/10（25 MHz，CE=2)
`100`	ATK-4384	800×480	同上
`101`	ATK-1018	1280×800	10/80/1280/70，3/10/800/10（50 MHz，CE=1)

★ 为什么用"像素使能"而不是分频时钟：25 MHz 屏需要对 50 MHz 做 2 分频。 如果直接分频当时钟用，会产生非全局网络的衍生时钟（要额外 `BUFG`），像素域与 系统域之间还要做跨时钟域。本程序改成 `pixel_en` 门控：**全部逻辑仍在 50 MHz 单域**，只有"什么时候推进一个像素"被门控。

★ 这里有一个真实踩过的坑，写成了回归测试：`lcd_timing_gen` 里 **行计数受 `pixel_en` 门控，场计数只受"一行结束"门控。**如果两者一起被门控， 25 MHz 屏会被多拖一倍，帧率从 58.6 Hz 掉到 29.3 Hz，而屏幕上只是"暗了一点"， 极难发现。`tb_lcd_timing` 的测试 2 就是专门盯这条。

---

5. 10 幅测试画面

按键（板载 KEY，W9）短按切下一幅；不按则每 256 帧（约 4.4 s）自动轮播。

#	画面	看什么
0	彩条（白/黑/红/绿/蓝 5 等分）	RGB 三通道没接反、灰度台阶正常（与指南官方例程一致）

#	画面	看什么
1	红/绿/蓝/白/黑 单色轮换	该纯的地方纯不纯（偏色=某一通道权重错）
2	灰阶渐变 `R=G=B=0..255`	8 bit 灰度与伽马、有无台阶跳变
3	**1 像素棋盘**	像素级清晰度；糊=采样沿/相位不对
4	外框 + 128×64 网格 + 中心十字	有效区起点/终点、有无偏移或缩放
5	沿边框跑的小白块	刷新流畅度、有无撕裂
6	纯黑	背光漏光、亮点/暗点
7	纯白	亮度均匀性、暗角、电源压降
8	**触控可视化**：5 点十字准星 + 坐标/统计文本	触摸是否通信、坐标是否跟随
9	**10×10 触控响应格**	全屏覆盖、多点并发（第 1 点绿、第 2 点橙）

---

6. 触摸 IC 的两种协议

	GT911	FT5206 / FT5446
7 bit 地址	`0x5D`	`0x38`
寄存器地址宽度	**16 bit 大端**（先发高字节；指针 `0x814E` → 先 `0x81` 再 `0x4E`）	单字节
状态寄存器	`0x814E`：bit7 = 坐标就绪、bit6 = large detect、bit5 = 保留(0)、bit4 = have key、`[3:0]` = 点数	`0x02`（点数在 `[3:0]`）
坐标寄存器	`0x814F` 起，**每点 5 字节**：`{track id, X 低, X 高(仅低 4 位), Y 低, Y 高(仅低 4 位)}`	`0x03` 起，每点 4 字节：`{XH(仅低 4 位), XL, YH, YL}`
一次读多少	状态字 + 5 点 = **26 字节 = 13 个 16 bit 字**	8 字节
坐标拼法	`x = ((XH & 0x0F) << 8) | XL`	`x = ((XH & 0x0F) << 8) | XL`（主机先收到的字节在低位）
收尾	**必须写 `0x814E = 0` 清标记**（3 个写字节：`0x81`,`0x4E`,数据 0），否则不再更新	先读 `0x02` 再读 `0x03`，不能一条流水线读下来

★ GT911 的 5 字节/点布局有两个容易踩的坑，程序里都踩过并已修：

1. **点与点之间会跨越 16 bit 字的边界。**主机把先收到的字节放在低位，于是 `d0 = {track id0, status}`、`d1 = {X0H, X0L}`、`d2 = {Y0H, Y0L}`、`d3 = {X1L, track id1}`、`d4 = {X1H, Y1L}`、`d5 = {Y1H, track id2}` ..... 点 0 和点 2 落在字内，**点 1、3 跨字。** 早先按"每点 4 字节且字对齐"（`x1 = d3`）解码，点 1 以后全错（实测 `x1 = 0x0402` 而真值 `0x384`）。 正确写法：`x1 = {4'd0, d4[3:0], d3[15:8]}`、`y1 = {4'd0, d5[3:0], d4[15:8]}`，依次类推。
2. **探测判据不能用 `状态字 <= 5`。**状态字是 `0x82`（2 点就绪）时低 4 位才是点数、bit7 是"有数据"，`0x82` 明显大于 5 —— 判据过严就会"明明读到了却认不出 GT911"，每 500 ms 重探一次。 正确判据：**地址被应答 + 读回值 ≠ `0xFF`（总线空闲）+ bit5 == 0 + 点数 `[3:0] <= 5`。**

程序的探测顺序：先按 GT911 的 `0x5D` 读 `0x814E`，判据如上；否则试 FT 的 `0x02`， 再试 `0x03`。两者都不应答就每 500 ms 重探。 **只有锁定型号之后的失败才计入错误统计** —— 否则探测阶段"故意用错地址"刷出的

NACK 会污染指标（这是排查时最容易被误导的地方）。

IIC 速率：`SCL = 50 MHz / (4 × 62) = 201.6 kHz`，总线卡死看门狗 40 ms。 主机读缓冲 32 字节、支持最多 15 个 16 bit 字，足够 GT911 的 5 点全解析。

---

7. UART 遥测协议（115200 8N1）

每秒（按 58 帧门控，约 1.007 s）发一帧 **16 字节**，便于用电脑量化"到底跑了多少"， 而不是只靠肉眼：

当前是**协议版本 2**（`b2 = 0x02`）。上面机脚本 `lcd_uart_check.py` 对 v1/v2 都能解。

字节	含义
0 / 1 / 2	帧头 `0xA5` `0x4C`（`'L'`）/ 协议版本 `0x02`
3	`[7]id_valid`， `[4:0]` = 型号 one-hot（bit4=1018, bit3=7084, bit2=7016, bit1=4384, bit0=4342）★ 这里必须是 8 位拼接：曾经写成 9 位（`{id_valid,3'b000,5 个命中位}`）赋给 8 位 wire，**最高位被静默丢掉**，PC 侧永远看到 `id_valid=0`（型号位却是对的，很容易误判成"ID 没读到"）
4	`[6]int_pin [5]sda_line [4]scl_line`（三条在空闲时的实际电平；正常应全部为 1）， `[3:0]` 当前画面号 0..9
5	`[7]tch_valid`， `[1:0]tch_type`（1=GT911, 2=FT5x06, 0=未锁定）
6	**GT911 `0x814E` 原始状态字**：bit7 = 坐标就绪、bit6 = large detect、bit4 = have key、`[3:0]` = 点数
7	GT911 产品 ID 首字节（**`0x39` = `'9'`**；读到 `0x00` 说明总线被拉死/器件不在工作态，而不是"没有触摸"）
8 / 9	第 1 点 x（高字节 / 低字节）
10 / 11	第 1 点 y
12 / 13	芯片内配置的 **X 分辨率**（`0x8146/0x8147`；7 寸屏应为 1024）
14 / 15	芯片内配置的 **Y 分辨率**（`0x8148/0x8149`；应为 600）

拿这一帧的几个数字就能判断触摸是否健康，判读顺序：

1. `int_pin` / `sda_line` / `scl_line` 是否都是 1 ⇒ 电气层（开漏上拉、FPC 接触）。
2. `b5[1:0]` 是否锁成 1(GT911) ⇒ IIC 协议层（读指针、地址、ACK）。
3. `b7` 是否 `0x39` ⇒ 器件真的在应答并能读回真数据。
4. `b12/13`、 `b14/15` 是否 `1024` / `600` ⇒ **芯片内部配置表是否已加载。** 读到 0 说明芯片的配置区没被写进去（GT911 的应用固件/配置不跑）， 这时 `0x814E` 自然恒为 0、点数恒为 0 —— 与"能读产品 ID 但不上报触摸"完全一致。
5. 最后才看 `b6` 的 bit7 与点数、以及按屏时坐标是否变化。

---

8. 怎么跑

```
# 1) 仿真（五个测试台：时序 / UART 帧级 / 触摸-GT911 / 触摸-FT5206 / 集成；约 5 分钟）
powershell -NoProfile -ExecutionPolicy Bypass -File tools\run_vivado.ps1 `
    -Tcl lcd_test\tools\sim_lcd_test.tcl -LogFile _lcd_work\sim_lcd.log -Tail 200
# 在 _lcd_work\sim_lcd.log 里 grep "TEST PASSED" / "TEST FAILED"
# 每个测试台的 stdout 另存在 _lcd_work/xsim_lcd/*_run.log

# 2) 综合验证（含双向 IOBUF 检查）
powershell -NoProfile -ExecutionPolicy Bypass -File tools\run_vivado.ps1 `
    -Tcl lcd_test\tools\synth_lcd_test.tcl -LogFile _lcd_work\synth_lcd.log -Tail 60

# 3) 上板：实现 + 出位流
powershell -NoProfile -ExecutionPolicy Bypass -File tools\run_vivado.ps1 `
    -Tcl lcd_test\tools\impl_lcd_test.tcl -LogFile _lcd_work\impl_lcd.log -Tail 60
# 产物：_lcd_work/impl_lcd/lcd_test_top.bit

# 4) 上板：经 JTAG 下载（只写易失配置，不动 SPI flash）
powershell -NoProfile -ExecutionPolicy Bypass -File tools\run_vivado.ps1 `
    -Tcl lcd_test\tools\program_lcd_test.tcl -LogFile _lcd_work\program.log -Tail 40

# 5) 上板：读 UART 遥测（115200 8N1，需要 E:\Anaconda\python.exe，它带 pyserial）
& E:\Anaconda\python.exe lcd_test\tools\lcd_uart_check.py --port COM3 --seconds 6
```

> `tools/run_vivado.ps1` 会设好 PATH、把 `HOME/TEMP/LOCALAPPDATA` 钉在工作区内、 并把 `XILINXD_LICENSE_FILE` 指到 GOLD 许可（不设则 ILA 相关命令会报 `ERROR: [Vivado 12-29205] ... current selected license is BASIC`）。

---

9. 上板步骤与判据

1. **先插排线、后上电。**FPC 座是下接式，插反等于把 +5V 灌进数据线，**会烧屏。** 插好后用万用表确认 J7 第 1/2 针有 +5V、第 11/20/29 针是 GND（这三针的接地是 按排法推定的，值得实测）。
2. 下载 `lcd_test_top.bit`。屏幕应立刻出现**彩条**（画面 0），背光同时亮。 全黑先查 `lcd_bl`(Y2) 与 +5V。
3. 刷新率应当是 **58.6 Hz**（`50 MHz ÷ (635 × 1344)`）。用手机慢动作对着 画面 7（纯白）拍，能看到轻微横纹但不闪。
4. 依次按键过 10 幅画面，重点看 **画面 3（1 像素棋盘）** 与 **画面 4（网格）**。
5. 打开串口工具（115200 8N1），确认每帧 `0xA5 4C`。看字节 3 的型号位： 本屏应为 `0x04`（bit2 = 7016）。
6. 摸屏：**画面 8** 应出现彩色十字准星与坐标；**画面 9** 应点亮对应大格， 两指同时按应有两个不同颜色的格子同时亮。
7. 若触摸一直不通信：先确认 `touch_rst_n`(W4) 上电为高、 `touch_int`(T5) 空闲为高， 再看遥测的 `err_ps` 是"全是 NACK"（地址/上拉问题）还是"全是超时"（SCL 被拉死）。

10. 已知边界（诚实说明）

- `J7` 针 11/20/29 的接地、针 1/2 的 +5V 是**按排法推定**的，本程序没有、也无法 在 FPGA 内部验证它们 —— 上板前用万用表点一下。
- ATK-4342 / ATK-1018 的时序用的是 VESA 标准值兜底，本程序的目标屏是 ATK-7016； 另外两款只保证"能点亮"，不保证刷新率精确。
- 触摸 IC 的 INT 引脚只做上报与计数，**不参与触发**（轮询式读取足够， 且能顺带量化 IIC 负载）。
- 本程序不涉及 BiSS/SPI/编码器任何既有功能，两个顶层永远分开综合。
- `lcd_bl` 直接接常数 1（背光常开），综合器会报 `WARNING: [Synth 8-3917] port lcd_bl driven by constant 1` —— 是设计意图。
- 触摸控制器里的 `gt_status` / `int_low_cnt` 两个统计寄存器没有被遥测帧取用， 综合时会被优化掉（`WARNING: [Synth 8-6014] Unused sequential element ... removed`）。 它们只在仿真里被测试台读取；上板想用就把它们加进遥测帧。
- 布线后的 DRC 有两条**良性**警告，都不影响功能：
- `RPBF-3 IO port buffering is incomplete`（21 条）—— 那 21 根只输出的 `lcd_rgb` 位没有输入缓冲。原因是"24 位 inout 总线里只有 3 位（R7/G7/B7） 需要读回"；ID 窗口里 `data_oe=0` 会让它们全部高阻，功能上正确。
- `BUFC-1 Input Buffer Connections`（1 条）—— 同类提示。 想让报告完全干净，可以在 xdc 里加 `create_drc_waiver -id RPBF-3 -description "lcd_rgb 只作输出，无需输入缓冲"`； 本程序选择保留警告并在这里说明，避免把真问题一起豁免掉。

---

11. 验证记录（真实 Vivado 2026.1，`xc7a35tfgg484-2`）

11.1 仿真：4 个测试台全部 `TEST PASSED: 0 errors`

测试台	配置	关键实测
`tb_lcd_timing`	ATK-7016 参数	行周期 `1344` 像素、`de` /行 `1024`、`de` /帧 `614400`、行数 `635`、帧周期 `853440` 拍（**58.59 Hz**）； `pixel_en` 降到 1/2 后帧周期变 `1706880` 拍（**29.29 Hz**）——后者是"场计数没有被像素使能门控"的回归判据
`tb_touch_i2c`（GT911 假从机）	`SLV_ADDR=0x5D`	锁定 `tch_type=1`、点数 2、`x0/y0 = 600/256`、`x1/y1 = 900/400`、`stat_reads=3`、`nack=0`、`timeout=0`
`tb_touch_i2c`（FT5206 假从机）	`xvlog -d SLAVE_FT5206`	锁定 `tch_type=2`、点数 1、`x0/y0 = 258/772`、`stat_reads=3`、`nack=0` —— 验证 **GT 探测失败后能立刻转 FT 探测**（`slow_cnt` 重新置位那处修复的回归）
`tb_lcd_rgb_top`（集成）	屏侧电阻 + GT911	`panel_id=010`、ID 采样瞬间 `lcd_rgb=7fff7f`、`data_oe` 低电平 `10486` 拍、行 `1344` /行同步 `20` /场同步 `4032`（=3 行）/每帧 `614400` 像素、触摸锁定 + 坐标 `600/256`、按键使画面 `0→1`、UART 高电平 `434` 拍（115200）

跑法见第 8 节；三个测试台的 stdout 分别落在 `_lcd_work/xsim_lcd/*_run.log`。

11.2 综合（`_lcd_work/synth_lcd/`）

- `synth_design completed successfully`：**0 errors / 0 critical warnings**， `All user specified timing constraints are met.`
- **双向引脚只有 5 个 IOBUF**：`lcd_rgb[7]` / `[15]` / `[23]`（读屏 ID 的 R7/G7/B7）

- `touch_scl` / `touch_sda`。其余 21 根数据线只输出，综合器把它们做成 `OBUFT`（`IBUF` / `OBUF` / `OBUFT` = 2/8/21）—— 这是正确的，也说明"读 ID 只需要 3 根线能读"。
- 资源：LUT 4211（LUT1 366 + LUT2 789 + LUT3 480 + LUT4 1100 + LUT5 434 + LUT6 1042）、 触发器 770（FDCE 725 + FDPE 24 + FDRE 21）、CARRY4 641、BUFG 1。 对比板上原编码器工程（11k+ 端点）非常小，完全不影响。
- 3 条良性警告见第 10 节。

11.3 实现 + 位流（`_lcd_work/impl_lcd/`）

`opt_design → place_design → phys_opt_design → route_design → write_bitstream` 全部成功：

- 布线后 `All user specified timing constraints are met.` `sys_clk` **WNS = +0.581 ns** / TNS 0.000 / **0 个失败端点**（2180 端点）； 保持 WHS = +0.084 ns；脉冲宽度 WPWS = 9.500 ns。
- DRC：**0 Errors / 0 Critical Warnings**，只有两条良性警告（见下）。
- 资源（布线后）：Slice LUT 3196（15.37%）、Slice Register 771（1.85%）、 Block RAM 0、DSP 0、Bonded IOB 36（14.40%）。
- 产物 `_lcd_work/impl_lcd/lcd_test_top.bit`： **507,773 字节**，2026-10-14 19:38:34， SHA256 `5031A2430D4E83CBF608DA1983EAB1FDA40DDA0C2168192D57B112658246A2E6`。

上板前请重新跑一遍第 8 节的 2)+3)（RTL 若有改动，位流哈希会变）。

---

12. 上板实测记录（2026-10-14，达芬奇-lite + ATK-7016）

连法与工具：JTAG `localhost:3121/xilinx_tcf/Digilent/210512180081`（器件 `xc7a35t_0`，IDCODE `00000011011000101101000010010011`）；串口 `COM3 @115200`。 下载用 `lcd_test/tools/program_lcd_test.tcl`（**只写易失配置，不动 SPI flash** —— 断电后板子仍回到 flash 里的编码器生产位流）。读遥测用 `lcd_test/tools/lcd_uart_check.py`（要带 pyserial 的 `E:\Anaconda\python.exe`）。

> ⚠ 下载这一步需要**放开文件沙箱**：受限模式下 Vivado 的 JTAG 服务（`cs_server`）无法在临时目录里解包 `VCRUNTIME140.dll`，会报 `ERROR: [Labtools 27-2220] Unable to launch local cs_server executable.` + `Failed to extract VCRUNTIME140.dll: ... Permission denied`。原因是 PyInstaller 建的一次性 `_MEIxxxxxx` 目录自带保护 ACL，只给交互用户授权，沙箱身份进不去。 放开权限跑一次即可，不需要改任何 ACL。

实测结论

项	结果	判定
屏幕画面	彩条 / 整屏单色在自动轮换（用户目视确认）	✅ 时序、24 位数据线、背光全通
UART 遥测	8 秒收到 8 帧（严格 1.007 s 一帧），帧头 `A5 4C`	✅ 修掉每字节多发一位的 bug 后完全正常
屏 ID	`id_bits = 0x04` ⇒ 型号 one-hot 命中 **7016**（`{B7,G7,R7}=010`）	✅ 模块上的 ID 分压真的读到了：24 根数据线的双向 + 高阻窗口在真机上成立

项	结果	判定
触摸型号	`tch_type = 1`（**GT911**），`prod_id = 0x39`（`'9'`），`err_ps = 0`	✅ 屏上触摸 IC 就是 GT911，地址 0x5D 被正确应答且能读回真实寄存器内容（★ 早期"锁定 GT911"是 IOBUF 极性反相造成的**假锁定**，见下第 4 条；现在由 `prod_id = 0x39` 独立证实）
IIC 速率	约 `300` 次读/秒（201.6 kHz SCL，每次 26 字节）	✅ 合理
IIC 空闲电平	`sda_line = 1`、`scl_line = 1`	✅ 总线健康，没有被器件压死
触摸坐标	**用户实际按住屏幕时 `tch_num` 仍为 0**（21 帧里 0 帧有点）	❌ 待查，见下

只有上板才暴露出来的三个问题（都已修）

1. **UART 每字节多发一位**：`uart_tx_byte` 的位选择写成了 `0..6 → shreg[0..6]`、`default → shreg[7]`，而"停止位周期"落进了同一个 `default`，于是每字节被发成 11 位（起始 + 8 数据 + **重复的 MSB** + 停止）。 MSB=0 的字节在 PC 侧变成"停止位读成低"⇒ 帧被截断、字节错位 （现场实测 16 字节只收到 7 个）。修法：`4'd7 → shreg[7]`、`default → 1'b1`。 **教训**：原来集成台只数了"UART 有起始位"，从没逐字节解码过一帧， 所以这个 bug 在 4 个测试台全绿的情况下照样上板炸掉。现已补 `sim/tb_uart_frame.v`（8N1 逐位采样解码 16 字节，修前 13/16 字节报 framing error， 修后 `framing_errors=0 bytes_received=16`）。
2. **`id_bits` 拼接位宽错**：写成 9 位赋给 8 位 wire，`id_valid` 被静默丢掉， PC 侧一直显示 `panel id valid: 0`（型号位却是对的，极易误判成"ID 没读到"）。
3. **`reads_ps` 报的是累计值**：`stat_reads` 只增不减，直接快照会一路涨到 65535 再回绕。现已改成**每秒差值**。
4. **★ `i2c_io_wrap` 里 IOBUF 的 `T` 极性接反**（最隐蔽的一个）：本工程的约定是 `oe = 1` 表示**拉低**，代码于是直接写 `.T(scl_oe)` / `.T(sda_oe)`；而真实 Xilinx `IOBUF` 的 `T` 是**高有效三态使能**（`T=1` = 高阻、`T=0` = 驱动）——极性正好相反。 上板后果是"想拉低变成释放、想释放变成拉低"，SDA 被自己恒拉低 ⇒ 任何地址都像 被 ACK、读回全 `0x00` ⇒ 探测判据（地址被 ACK + 字节 ≠ 0xFF + 点数 ≤ 5）**假通过**， 于是"锁成 GT911 但点数恒 0、`err_ps = 0`"。 **为什么 5 个测试台全绿也没抓到**：仿真桩 `sim/iobuf_stub.v` 当时写的是 `assign IO = T ? I : 1'bz;`（`T=1` 驱动）——**与真实原语相反**，两个错误互相抵消， 仿真永远是"对"的。修法是把两处都改成正确极性（RTL 用 `.T(~oe)`，桩改成 `IO = T ? 1'bz : I`），两次反相抵消后仿真行为不变，仍 5 个测试台全过。 **教训**：凡是"把某些东西反一下"的改动，必须同时对**仿真模型**和**真实原语**负责； 仿真桩与真实原语反向，等于把被测逻辑的真错藏起来。

待查：触摸点数为 0（当前唯一未通过项）

现象：型号锁定为 GT911、`err_ps = 0`、每秒 ~300 次读全部成功，但用户按住屏幕时 状态字低 4 位恒为 0。已用两条诊断把范围缩到最小：

- **b7 = GT911 产品 ID 首字节**（读 `0x8140`）：现场实测 **`0x39` = `'9'`** —— 真片在应答， 且读回的是真实数据（不是总线被拉死时的 `0x00`）。
- **b4 bit6/bit5/bit4 = INT / SDA / SCL 空闲电平**：实测 **1 / 1 / 1** —— 总线健康。
- 由此排除电气层与 IIC 协议层，问题在**芯片内部为什么不上报。**

已按官方时序补的两处（待新位流验证）

1. **INT 开漏需要上拉 + 上电时序。**GT911 的 INT 是开漏输出，官方上电时序是 `RST 低` 期间 `INT 低`（地址在 RST 上升沿按 INT 电平锁存：0x5D / 0x14）→ `RST 释放后 INT 再保持低约 50ms`（让芯片装载配置）→ 之后 `INT` 必须能回到高电平， 芯片才用低脉冲表示"有数据就绪"。模块的 INT 在 FPC 上不一定有上拉电阻， 而本程序原来把 `touch_int` 当纯输入、既不驱动也不上拉。现在： `touch_int` 改为 `inout`，复位期间与本程序自己算的 50 ms 窗口内由 FPGA 拉低， 之后转高阻；约束里给该脚加 `PULLTYPE PULLUP`（内部弱上拉）， 遥测 b4 bit6 上报释放后的真实电平。
2. **读芯片内部配置。**新增 `STEP_GT_INFO`：读 `0x8144/0x8145`（固件版本）与 `0x8146..0x8149`（**配置的 X/Y 分辨率**，7 寸屏应为 1024/600）。 若读到 0，说明芯片的配置表没被写进去、应用固件没跑起来 —— 那就需要按 Goodix 的流程把配置表写到 `0x8047` 起，再写 `0x8100` 的 soft reset 生效。 同时 b6 直接上报 `0x814E` 的**原始字节**（看 bit7 到底有没有置起）。

判读顺序见 §7 末尾的 1→5 步。